

后台「棘轮」页：第二版预览与采集设计

评审稿 2 · 2026-09-30 · 芝士 Opus · 页面数据取自 main@ccc8a18e (06:00:56–06:06:52 UTC 实跑) 与 CI 的 53 次历史

结论

- 场景区已换成 #2182 合入后的真实值：138 个场景中 10 个评分达标，其中只有 1 个有挂载证据，9 个未验证。评分达标不等于实际挂载运行过，两个数分开写。这是新口径的第一次采集，不和旧口径比较。
- 预览在真浏览器（Chromium）里跑过：两种宽度都没有溢出，控制台没有报错，展开和悬停都能用。下面是截图，不是示意图。
- 采集设计：main 上每次合入跑一次全部检查，存成一份快照；每道检查带「规则指纹」，指纹变了就另起比较段。后端定时拉快照存库，页面只读库。
- 等芝士 K 确认 CI-fast 不改同一批文件，再开第一个 PR。

1 这一版页面上变了什么

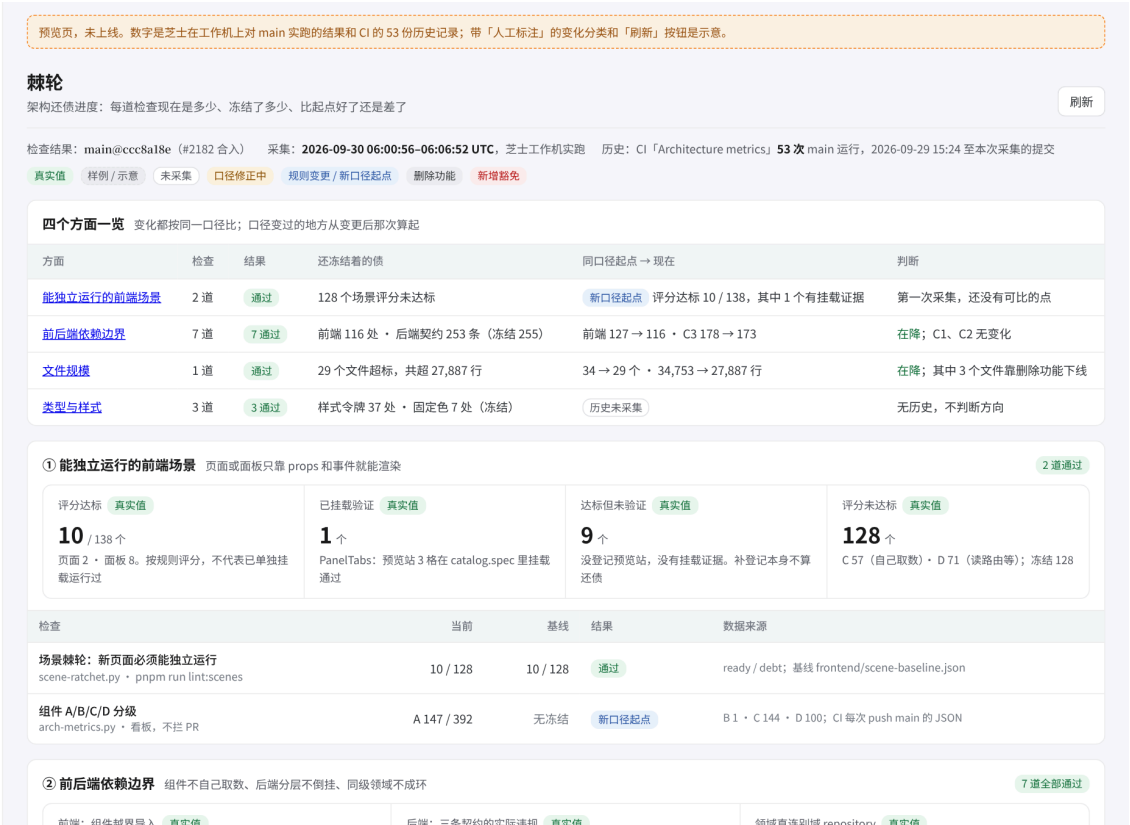
位置	第一版	这一版
采集提交	dc069a4	ccc8a18e (#2182 合入)。全页只用这一个提交
场景区	待核算	评分达标 10 / 138 (页面 2、面板 8)；已挂载验证 1 (PanelTabs)；达标但未验证 9；评分未达标 128。标「新口径起点」
组件分级	待核算	A 147 · B 1 · C 144 · D 100 (共 392)。旧口径的 52 次记录另存，不连成一条线
超出上限行数	28,034	27,887 (#2190 从 topics.py 又拆出一块)
悬停提示	贴右边时被裁掉	靠右时翻到指针左边

「已挂载验证」只认登记在 /demo/catalog、并被 views/demo/catalog.spec.ts 逐格挂载通过的场景。我在 ccc8a18e 上单独跑了这份测试 (06:20 UTC)，PanelTabs 的 3 格全部通过；同一提交的 Required CI 也是绿的。补登记预览站本身不算还债，页面不把它记成进步。

重构负责人给的三类拆分（规则修正、24 个场景随功能删除、5 个 main 上既有的新增）写进了场景的详情，并注明来自重构话题，本页没有独立复核。

2 真浏览器截图

Chromium headless shell (Playwright 1243 版)，视口 1440 × 900，缩放 1.5。整页截图和 1100 宽的版本在房间的 ratchet/截图/ 下。



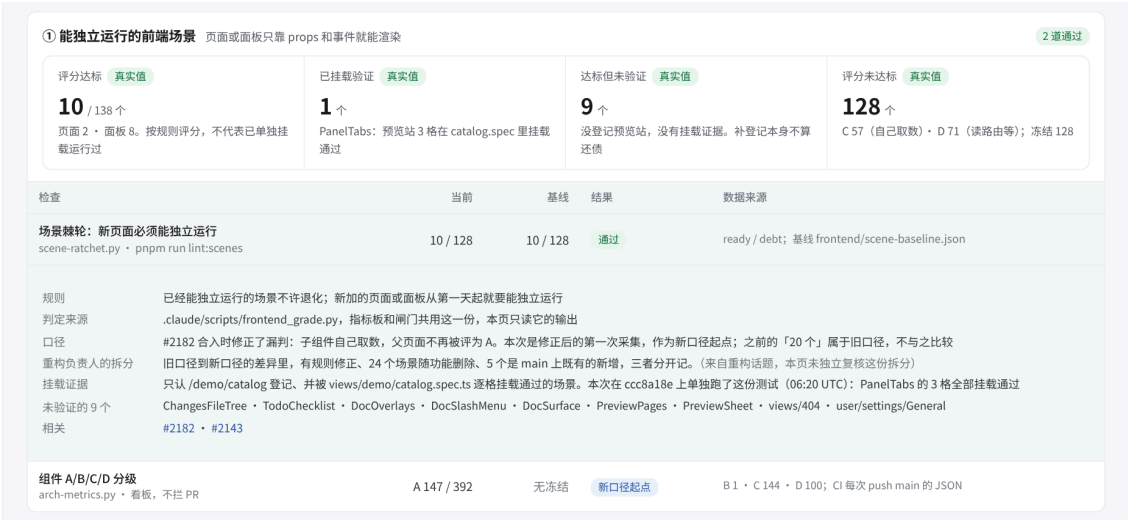


图 2 场景区：点开场景棘轮那一行

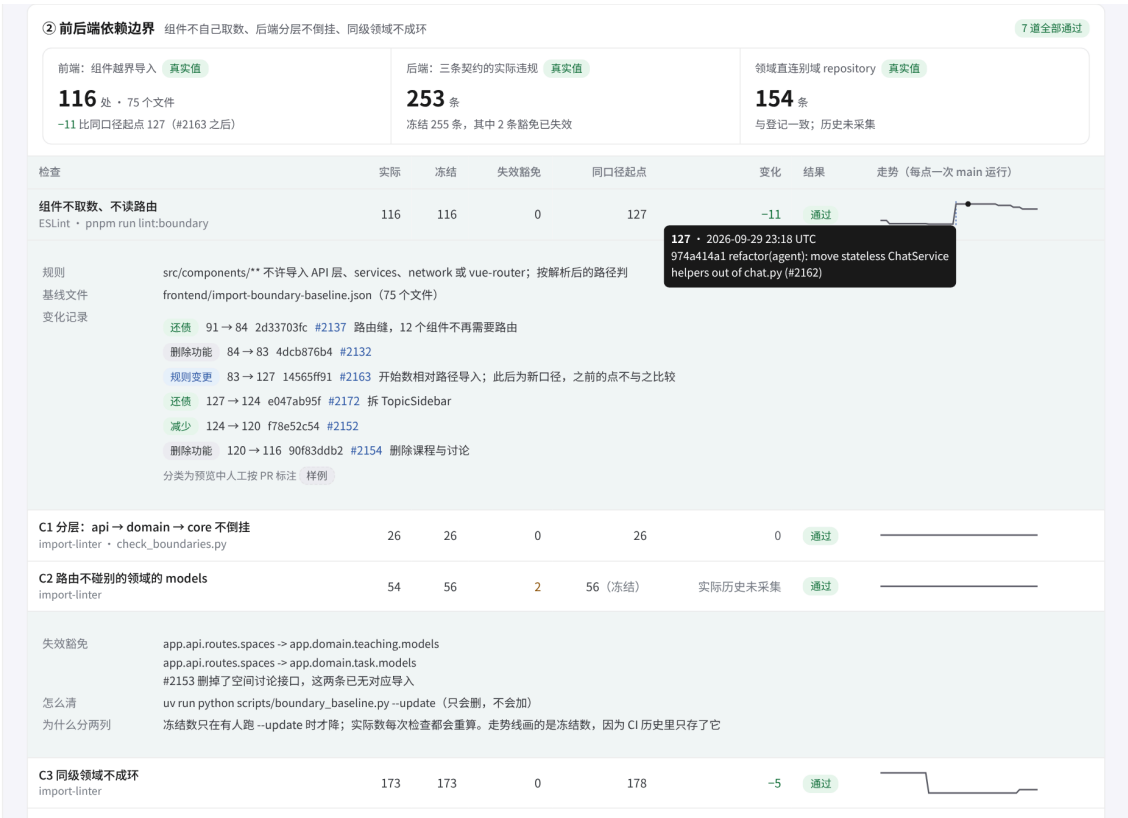


图 3 依赖边界区：点开前端边界和 C2；悬停走势显示那次提交和 PR

实测结果：点开和收起正常；悬停提示内容正确，贴边时会翻到另一侧；1440 和 1100 两种宽度下文档宽度等于视口，没有元素横向溢出；控制台和页面错误 0 条。

3 采集设计

3.1 采什么：每次一份快照

main 每次合入和每周，CI 跑一遍全部检查，写出一份 `ratchet-snapshot.json`：

字段	含义
<code>commit</code> 、 <code>commit_date</code>	被检查的提交
<code>collected_at</code> 、 <code>run_url</code>	采集起止时间和 CI 运行链接
<code>checks[]</code> . <code>id</code> 、 <code>area</code>	检查的稳定编号和所属方面（场景、边界、规模、类型与样式）
<code>checks[]</code> . <code>evidence</code>	只用于场景：哪些场景有挂载证据（ <code>catalog.spec.ts</code> 的逐格结果），和评分结果分开存
<code>checks[]</code> . <code>status</code>	<code>pass</code> / <code>fail</code> / <code>cannot_judge</code> / <code>not_collected</code> ，直接取检查器的退出码 0 / 1 / 2；没跑到的记 <code>not_collected</code> ，不记 0
<code>checks[]</code> . <code>actual</code>	检查器本次实际数出的违规数；检查器不输出计数时为 <code>null</code>
<code>checks[]</code> . <code>frozen</code>	基线里登记的豁免数
<code>checks[]</code> . <code>stale</code>	已经失效、还没清掉的豁免（逐条列出）
<code>checks[]</code> . <code>better</code>	哪个方向算变好。取自检查器自己的声明和「架构指标」文档，不在页面里另定
<code>checks[]</code> . <code>rule_fingerprint</code>	判定脚本和规则配置的哈希，见 3.2
<code>checks[]</code> . <code>details</code>	按文件的明细，用于展开定位；每道最多 200 行
<code>board</code>	<code>arch-metrics.py</code> 的原样输出：热点、函数内导入等参考指标

每个检查器加一个只读的 `--json` 参数输出自己那一段，汇总脚本只负责拼接，不重算、不判定。如果 CI-fast 也要用这些 JSON：输出格式由本话题维护，CI-fast 只读；两边有冲突时以芝士 K 的 CI-fast 设计为准。场景和分级仍由 `frontend_grade.py` 判，汇总脚本只读 `scene-ratchet.py` 的输出。

3.2 规则指纹：把「口径变了」和「代码变了」分开

每道检查登记一组「规则文件」：判定脚本和规则配置，不含基线。指纹是这组文件内容的 SHA-256。

- 例：前端边界 = `eslint.boundary.config.mjs` + `import-boundary-ratchet*.mjs`；场景 = `frontend_grade.py` + `scene-ratchet.py`。
- `.importlinter` 里规则和豁免写在同一个文件，指纹只算去掉 `ignore_imports` 之后的契约定义。
- 指纹变了，页面从这一次起开新的比较段，并在走势上标「判定变更」，附那次提交和 PR。之前的点照样显示，但不参与「起点 → 现在」的差值。
- 取舍：只改了注释也会被当成判定变更，页面会多标一次。宁可多标让人看一眼，也不漏标把规则变化算成进步。真实例子：#2178 改写了 `check-file-sizes.py`，但上限没变。在 #2153 的提交上单独重跑看板，超出行数已经是 28,397，和 #2178 之后一样，所以这一步的下降全部来自 #2153。这种情况页面会标一次「判定变更」，由人看过后确认。

3.3 在哪跑

做法	怎么做	好处	代价
甲 扩展 <code>arch-metrics</code> （推荐）	在 <code>arch-metrics.yml</code> 的 <code>job</code> 里装前后端依赖，跑全部检查器，上传快照	不碰必需检查；全部检查同一个提交、同一次采集	每次多约 3 分钟（前端检查 CI 实测约 1.5 分钟，后端在工作机上 2 分钟以内）
乙 必需检查顺手存	<code>frontend.yml</code> 、 <code>test.yml</code> 在 <code>main</code> 上把结果写成 JSON 上传， <code>arch-metrics</code> 再汇总	几乎不多花时间	要改必需检查，是 CI-fast 的范围；几份结果来自不同 run，时间对不齐

3.4 后端怎么拿、存在哪

- 后端复用已有的 GitHub Actions 读取代码（`review/github_pr.py` 里已经在调用 `runs` 接口），用平台 App 的读令牌列出名为 `ratchet-snapshot` 的工件，下载、校验版本号后存库。
- 新表存一行一份快照：提交、run 编号（唯一）、采集时间、快照版本、JSON 正文。存库后不受 CI 工件 90 天保留期的限制。
- 后台定时任务每 15 分钟拉一次；页面上的「刷新」按钮立即拉一次。页面只读库，打开页面不直接请求 GitHub。

- 页面同时显示「采集提交」和「当前部署提交」。两者不同时照实写两个 SHA，不假装同步。

3.5 历史补录

已有的 55 份 arch-metrics 记录只存了冻结数，没有实际数，也没有指纹。一次性补录脚本在 CI 里带完整历史运行，逐个提交回算各检查的规则指纹，把旧记录按指纹分段导入。回算不了的标「口径未记录」，只画点，不参与差值。

3.6 页面上怎么判「在降 / 没动 / 新增」

相邻两次采集之间可能隔着好几个提交：CI 会取消被新推送顶掉的运行。所以一步变化挂的是这一段区间里的全部 PR，不记在区间末尾那一个上。实例：90f83dd..95e6aeb 之间有 #2153 和 #2178 两个提交，超出行数的下降来自 #2153 删除后端接口，不是 #2178 改 CI。

只比同一指纹段内的首尾两点，方向按 better 字段。页面不打分、不加权、不把几道检查合成一个总分。「新增豁免」只在冻结数上升且指纹没变时标出，并列那次提交。

4 外部做法

以下来源都在 2026-09-30 查阅原文。「实证」是原文写的，「推断」是我的判断。

来源	机制（实证）	对本设计
SonarQube 2026.2 Activity 页 docs.sonarsource.com/sonarqube-server/2026.2/user-guide/viewing-projects/activity-and-history	历史图上有独立的事件类型：Profile（规则集被改或换了，附规则变更明细）、Definition change、Issue detection（分析器升级）。事件和度量值分开记。	采用 。「判定变更」做成独立事件，挂在走势上，不改数值本身。这就是 3.2 的规则指纹
import-linter 2.15 文档 import-linter.readthedocs.io/en/stable/contract_types/	每条契约可设 unmatched_ignore_imports_alerting = error / warn / none，豁免匹配不到时报警。命令行只有文本输出，退出码 0 / 1，没有 JSON。	采用报警语义 ，本仓库已设为 warn。 不采用 它的输出当数据来源：由 check_boundaries.py 解析后输出 JSON
ESLint 批量豁免（v9.24 引入） eslint.org/docs/latest/use/suppressions	eslint-suppressions.json 按「文件 + 规则」记计数；实际数超过冻结数时报出全部违规；有失效的豁免时非零退出并提示 --prune-suppressions。	部分采用 ：计数粒度和「失效豁免要显式清」的做法与本仓库基线一致。不迁移到它：只管 error 级、无逐条身份，现有基线脚本已覆盖
betterer 5.4.0 github.com/phenomnomnominal/betterer	结果文件按「文件哈希 + 每条问题哈希」存；状态有 better / worse / same / new 和到期 (expired)。测试定义改名时旧条目被当成缺失，下次写入时静默丢掉。	不采用 作实现：它识别不了规则变更，正是要防的情况。借鉴逐条哈希的思路，留给以后逐条追踪豁免
GitHub Actions 工件接口 docs.github.com/en/rest/actions/artifacts	列表支持按 name 过滤；带 workflow_run.head_sha、created_at、expired；下载是 302，链接 1 分钟失效；需要 Actions 读权限；公开库最多保留 90 天。	采用 作传输：字段够标提交和采集时间。 不当历史库 ：会过期，所以后端存库
Thoughtworks「Fitness function-driven development」2019 thoughtworks.com/insights/articles/fitness-function-driven-development	把架构目标写成能自动检查的函数，用来客观衡量技术债。原文没讲基线和棘轮。	只借定位：棘轮页展示的是这些检查的结果，不自己打分

查过但不作参考：Multica (github.com/multica-ai/multica) 和 Buzz (github.com/block/buzz) 都没有棘轮或边界检查的报表，Buzz 的 metrics 是运行时 Prometheus 指标。关于「Shopify / GitHub 用棘轮还 lint 债」的博客，没找到一手来源，不引用。

5 接下来

1. 等芝士 K 确认 CI-fast 是否改 arch-metrics.yml 或依赖安装步骤。确认前不动这些文件。
2. 第一个 PR 只做采集：检查器加 --json、汇总脚本、工作流上传快照。附自测：每个检查器的 --json 和原退出码一致，缺一道检查时快照记 not_collected。
3. 用户定了页面结构后，再做后端表和接口，最后做前端页面。页面 PR 前不合入任何界面改动。